

Mathematical Notes for the Lemmas in `RefJasminNTT.ec`

Codex Proof Notes

28 April 2026

Abstract

These notes give a mathematician-style account of every lemma and equivalence proved in `RefJasminNTT.ec`. The `EasyCrypt` file is a refinement layer: it proves that the extracted Jasmin forward and inverse NTT cores refine the imperative reference procedures in `NTT_Fq.NTT`. The central mathematical themes are representation of signed machine words as field elements, Montgomery table cancellation, pointwise word-size bounds, and loop-level pRHL invariants. The presentation below follows the order of the `EasyCrypt` file and explains what each lemma proves and why the proof is valid.

Contents

1	Notation and Proof Obligations	2
2	Integer Shift Lemmas	2
3	Machine-Word Addition and Subtraction	2
4	Montgomery Constant and Table Cancellation	3
5	Montgomery Multiplication Lemmas	4
6	Pointwise Bound Infrastructure	5
7	Forward Stage Arithmetic	6
8	Forward Butterfly and Twiddle Lemmas	9
9	Inverse Stage Arithmetic	11
10	Inverse Butterfly and Twiddle Lemmas	13
11	Final Inverse Scaling Lemmas	14
12	Wrapper and Core pRHL Equivalences	16
13	Conclusion	17

1 Notation and Proof Obligations

All coefficients live in the finite field $\mathbf{F}_q = \mathbb{Z}/q\mathbb{Z}$, where the HAETAE modulus is $q = 64513$. A 32-bit machine word w is interpreted as the field element

$$\text{word}(w) = \text{incoeff}(\text{W32.to_sint}(w)).$$

For a byte array $rp : \text{BArray1024.t}$, the corresponding coefficient array is obtained pointwise by reading the i -th 32-bit lane:

$$\text{poly}(rp)[i] = \text{word}(\text{BArray1024.get32 } rp \ i).$$

The relation $\text{NTT_Fq.poly_repr}(rp, p)$ states $\text{poly}(rp) = p$. The bounded relation $\text{NTT_Fq.poly_repr_bound}(rp, p)$, additionally states that every lane of rp satisfies the signed s -bit word bound $\text{bw32}(-, s)$.

The proof file also uses a pointwise bound function $bd : \mathbb{Z} \rightarrow \mathbb{Z}$:

$$\text{barray256_bound_by}(rp, bd) \iff \forall i \in \{0, \dots, 255\}. \text{bw32}(\text{BArray1024.get32 } rp \ i, bd(i)).$$

This permits a loop invariant to say that already processed coefficients have one more bit of headroom than untouched coefficients.

2 Integer Shift Lemmas

int_shr1_div2. For every non-negative integer x , the EasyCrypt integer operation $x \text{ '}| \gg 1$ equals Euclidean division $x\%2$.

Proof. The operator $\text{'|} \gg$ is the integer right-shift notation expanded by EasyCrypt into the corresponding arithmetic division by a power of two. With shift amount 1, that power is 2. Since $x \geq 0$, Euclidean division and the intended non-negative shift semantics coincide.

int_shl1_mul2. For every integer x , the EasyCrypt integer operation $x \text{ '}\ll 1$ equals $2x$.

Proof. Expanding the definition of $\text{'}\ll$, left shift by one is multiplication by 2^1 . Hence the result is $x \cdot 2$.

3 Machine-Word Addition and Subtraction

word_to_coeff_add. Assume two words a, b satisfy bounds $\text{bw32}(a, a_s)$ and $\text{bw32}(b, b_s)$, with $0 \leq a_s, b_s < 31$. Then

$$\text{word}(a + b) = \text{word}(a) + \text{word}(b).$$

Proof. The hypotheses imply that the signed representatives of a and b lie in intervals contained in $(-2^{30}, 2^{30})$. Therefore their sum lies inside the range where W32.to_sint of the word addition is the integer sum of signed representatives. EasyCrypt's lemma $\text{W32.to_sintD_small}$ gives this equality at the integer level. Applying incoeff and using additivity of incoeff gives the field equality.

word_to_coeff_sub. Under the same hypotheses as the preceding lemma,

$$\text{word}(a - b) = \text{word}(a) - \text{word}(b).$$

Proof. The signed bounds again put both operands in a range where word subtraction does not wrap with respect to signed interpretation. The lemma $\text{W32.to_sintB_small}$ identifies the signed representative of $a - b$ with the integer difference. The conclusion follows from the compatibility of incoeff with subtraction.

forward_butterfly_bounds. If s and t satisfy bounds a_s and t_s , with both exponents in $[0, 30]$, then both butterfly outputs $s + t$ and $s - t$ satisfy bound $\max(a_s, t_s) + 1$.

Proof. The signed magnitude of each operand is bounded by the appropriate power of two. The sum and difference are therefore bounded by twice the larger bound, which is exactly one additional bit. The EasyCrypt proof invokes the word arithmetic lemmas `Fq.add_corr` and `Fq.sub_corr`.

inverse_butterfly_bounds. If t_0 and $coeff$ satisfy bounds a_s and b_s , then both $t_0 + coeff$ and $t_0 - coeff$ satisfy bound $\max(a_s, b_s) + 1$.

Proof. This is the same signed-interval argument as for `forward_butterfly_bounds`, specialized to the inverse butterfly temporaries. Addition and subtraction each cost at most one bit.

4 Montgomery Constant and Table Cancellation

unit_R_ref. The Montgomery constant $R = \text{NTT_Fq.R}$ is a unit of $\mathbf{F}_q$.

Proof. The constant R is the residue of the Montgomery radix modulo q . The proof expands R , reduces the claim to a non-zero residue modulo q , and uses the unit criterion for the field $\mathbf{F}_q$.

inv_R_ref. The inverse of R in $\mathbf{F}_q$ is `incoeff(50386)`.

Proof. Since R is a unit by `unit_R_ref`, it suffices to multiply both sides by R . Expanding R and reducing modulo q , EasyCrypt checks

$$R \cdot 50386 \equiv 1 \pmod{q}.$$

The field equality follows from the injective residue equality used by `incoeff`.

array256_mont_get. For $i \in \{0, \dots, 255\}$,

$$\text{NTT_Fq.array256_mont}(p)[i] = p[i] \cdot R.$$

Proof. The operator `array256_mont` is an `Array256.map/init` definition that multiplies every coefficient by R . Expanding the definition and evaluating the initializer at an in-range index proves the claim.

mont_cancel. For $i \in \{0, \dots, 255\}$,

$$\text{NTT_Fq.array256_mont}(p)[i] \cdot R^{-1} = p[i].$$

Proof. By `array256_mont_get`, the left-hand side is $p[i] \cdot R \cdot R^{-1}$. The lemma `inv_R_ref` identifies R^{-1} , and the unit law $RR^{-1} = 1$ reduces the expression to $p[i]$.

jzetas_get_cancel. For $1 \leq i < 256$,

$$\text{word}(\text{jzetas}[i]) \cdot R^{-1} = \text{NTT_Fq.zetas}[i].$$

Proof. The table lemma `NTT_Fq.jzetas_get` states that the extracted table entry at index i is the Montgomery encoding of the abstract forward twiddle: $\text{word}(\text{jzetas}[i]) = (\text{array256_mont zetas})[i]$. Applying `mont_cancel` cancels the factor R .

array256_mont_inv_get. For $i \in \{0, \dots, 255\}$,

$$\text{NTT_Fq.array256_mont_inv}(p)[i] = p[i] \cdot R.$$

Proof. This is the inverse-table analogue of **array256_mont_get**. It follows by expanding **array256_mont_inv** as the pointwise multiplication map.

mont_inv_cancel. For $i \in \{0, \dots, 255\}$,

$$\text{NTT_Fq.array256_mont_inv}(p)[i] \cdot R^{-1} = p[i].$$

Proof. Combine **array256_mont_inv_get** with the identity $RR^{-1} = 1$, using **inv_R_ref**.

jzetas_inv_get_cancel. For $i \in \{0, \dots, 254\}$,

$$\text{word}(\text{jzetas_inv}[i]) \cdot R^{-1} = \text{NTT_Fq.zetas_inv}[i].$$

Proof. The imported lemma **NTT_Fq.jzetas_inv_get** identifies the concrete inverse table entry with the Montgomery encoding of **zetas_inv**[i]. The cancellation lemma **mont_inv_cancel** removes R . The range stops at 254 because index 255 is the special final scaling constant.

jzetas_inv_255_scale_cancel. At the final inverse-table index,

$$\text{word}(\text{jzetas_inv}[255]) \cdot R^{-1} = \text{NTT_Fq.scale255} \cdot R.$$

Proof. This is exactly the table fact **NTT_Fq.jzetas_inv_255_scaleE**. It states that the extracted word at index 255 is chosen so that, after one Montgomery multiplication, the effect is multiplication by **scale255** and retention of one Montgomery factor.

5 Montgomery Multiplication Lemmas

fqmul_bw32_16. If the integer product $aa \cdot bb$ lies in the signed Montgomery reducer range

$$-\frac{R}{2}q \leq aa \cdot bb < \frac{R}{2}q,$$

then a call to **Hpoly_extract.M.__fqmul** with operands whose signed values are aa and bb returns a 16-bit bounded word.

Proof. The extracted multiplication routine is related to the signed Montgomery reducer by **Fq.fqmul_corr_h**. The reducer correctness lemma **Fq.SignedReductions.SREDCp_corr** says the result lies in $(-q, q)$. Since $q < 2^{16}$, the returned signed word satisfies **bw32**($-, 16$).

fqmul_word_to_coeff_mul. Under the same product-range hypothesis, **__fqmul** returns a word whose field interpretation is

$$\text{incoeff}(aa) \text{incoeff}(bb) R^{-1}.$$

Proof. Again use **Fq.fqmul_corr_h** to replace the procedure result by the signed Montgomery reduction of $aa \cdot bb$. The congruence part of **SREDCp_corr** states that this reduction is congruent to $aa \cdot bb R^{-1}$ modulo q . Translating through **incoeff** and **inv_R_ref** gives the field equality.

fqmul_word_to_coeff_mul_h. This is the Hoare-form version of `fqmul_word_to_coeff_mul`, with the product-range hypothesis placed in the procedure precondition.

Proof. The proof is the same congruence argument as above. The only difference is logical: the product bound is destructed from the Hoare precondition rather than supplied as a lemma parameter.

fqmul_word_to_coeff_mul_bound_h. This combines the semantic conclusion of `fqmul_word_to_coeff_mul_h` with the 16-bit output bound of `fqmul_bw32_16`.

Proof. Use Hoare consequence twice. The first consequence supplies the field-value postcondition from `fqmul_word_to_coeff_mul_h`; the second invokes `Fq.fqmul_corr_h` and `SREDCp_corr` to obtain the 16-bit bound.

fqmul_word_to_coeff_mul_bound_ph. The probabilistic Hoare version of the previous lemma holds with probability one.

Proof. The routine `__fqmul` is lossless by `Fq.fqmul_ll`. Combining losslessness with the ordinary Hoare triple `fqmul_word_to_coeff_mul_bound_h` yields the probability-one `phoare` statement.

6 Pointwise Bound Infrastructure

The file defines

$$\text{barray256_bound_by}(rp, bd)$$

and

$$\text{poly_repr_bound_by}(rp, p, bd).$$

The first is a pointwise signed-word bound for all 256 lanes; the second adds the field representation relation. The following lemmas are the elementary logic of these definitions.

barray256_bound_by_get. If `barray256_bound_by(rp, bd)` holds and $i \in \{0, \dots, 255\}$, then the i -th lane satisfies `bw32(get32 rp i, bd(i))`.

Proof. This is immediate from universal elimination in the definition of `barray256_bound_by`.

barray256_bound_by_weaken. If $bd_1(i) \leq bd_2(i)$ for every in-range i , then a buffer bounded by bd_1 is also bounded by bd_2 .

Proof. At each index, use `NTT_Fq.bw32_weaken`, which says that a signed word bounded by a smaller exponent is bounded by a larger exponent.

barray256_bound_by_set32. If rp is bounded by bd , i is in range, and the new word w satisfies `bw32(w, bd(i))`, then `set32 rp i w` is still bounded by bd .

Proof. For an arbitrary lane j , use the byte-array get-after-set lemma: the read is w when $j = i$, and is the old lane otherwise. The case $j = i$ uses the hypothesis on w ; the other case uses the old bound.

barray256_bound_by_set32_change. Suppose rp is bounded by bd . If a store at i writes a word bounded by $bd'(i)$, and every untouched lane has $bd(k) \leq bd'(k)$, then the updated buffer is bounded by bd' .

Proof. The same get-after-set case split is used. At the updated index, the new word has the desired bound by hypothesis. At all other indices, the old bound is weakened from $bd(k)$ to $bd'(k)$.

poly_repr_bound_by_repr. The relation `poly_repr_bound_by(rp, p, bd)` implies `NTT_Fq.poly_repr(rp, p)`.
Proof. This is the first conjunct of the definition.

poly_repr_bound_by_bound. The relation `poly_repr_bound_by(rp, p, bd)` implies `barray256_bound_by(rp, bd)`.
Proof. This is the second conjunct of the definition.

fqmul_product_bound_16_24. If a is 16-bit bounded and b is 24-bit bounded, then

$$-\frac{R}{2}q \leq \text{W32.to_sint}(a)\text{W32.to_sint}(b) < \frac{R}{2}q.$$

Proof. The signed bounds give $|\text{W32.to_sint}(a)| < 2^{16}$ and $|\text{W32.to_sint}(b)| < 2^{24}$. With the concrete HAETAE values of R and q , the product interval is strictly contained in the Montgomery reducer input interval. EasyCrypt discharges the remaining integer arithmetic by normalization and SMT.

jzetas_bound16. Every forward twiddle table entry `jzetas[i]`, for $1 \leq i < 256$, is 16-bit bounded.

Proof. Expand the concrete list `Hpoly_extract.jzetas` and use `BArray1024.get32_of_list32`. Each of the 255 in-range constants is checked by reducing the signed word conversion and verifying its magnitude is below 2^{16} .

jzetas_inv_bound16. Every inverse twiddle table entry `jzetas_inv[i]`, for $0 \leq i < 256$, is 16-bit bounded.

Proof. The proof is the same finite table enumeration as `jzetas_bound16`, now over all 256 entries of the inverse table, including the final scaling entry.

barray256_bound_by_const. A uniform bound `NTT_Fq.barray256_bound(rp, s)` implies the pointwise bound by the constant function $i \mapsto s$.

Proof. Both definitions quantify over the same 256 indices. Expanding `const_bound` makes the conclusions identical.

barray256_bound_by_const_bound. Conversely, a pointwise bound by the constant function $i \mapsto s$ implies `NTT_Fq.barray256_bound(rp, s)`.

Proof. Expand `const_bound`; the pointwise and uniform definitions coincide.

7 Forward Stage Arithmetic

The forward stage definitions encode the descending layer sequence:

$$128, 64, 32, 16, 8, 4, 2, 1, 0.$$

The associated bound table is

ℓ	128	64	32	16	8	4	2	1	0
<code>fwd_stage_bound</code> (ℓ)	16	17	18	19	20	21	22	23	24

and the twiddle base table is

ℓ	128	64	32	16	8	4	2	1	0
<code>fwd_zbase</code> (ℓ)	0	1	3	7	15	31	63	127	255.

fwd_stage_bound_range. For every legal forward length, $\text{fwd_stage_bound}(\ell)$ lies between 0 and 24.

Proof. The predicate fwd_len_ok is a finite disjunction over the nine legal values. Checking the definition of fwd_stage_bound on each case gives one of 16, ..., 24.

fwd_stage_bound_pos. For every legal forward length, $\text{fwd_stage_bound}(\ell) \geq 0$.

Proof. This is the lower half of $\text{fwd_stage_bound_range}$.

fwd_stage_bound_lt31. For every legal forward length, $\text{fwd_stage_bound}(\ell) < 31$.

Proof. The upper half of $\text{fwd_stage_bound_range}$ gives at most 24, hence strictly less than 31.

fwd_stage_bound_ge16. For every legal forward length, $\text{fwd_stage_bound}(\ell) \geq 16$.

Proof. Case analysis on the finite definition gives values 16, ..., 24.

fwd_stage_next. If ℓ is a positive legal forward length, then

$$\text{fwd_stage_bound}(\ell/2) = \text{fwd_stage_bound}(\ell) + 1.$$

Proof. The positive legal lengths are 128, ..., 1. Halving moves one step to the right in the bound table, increasing the bound by one.

fwd_len_next_ok. If ℓ is a positive legal forward length, then $\ell/2$ is again a legal forward length.

Proof. The allowed positive lengths are powers of two from 128 to 1. Dividing by two gives the next entry in the list, with $1/2 = 0$ in integer division, which is included as the loop-exit length.

fwd_zbase_next. For a positive legal length,

$$\text{fwd_zbase}(\ell/2) = \text{fwd_zbase}(\ell) + 256/(2\ell).$$

Proof. Each forward layer consumes $256/(2\ell)$ twiddle entries. The explicit fwd_zbase table records the cumulative number of entries consumed before the layer. Case analysis over $\ell = 128, \dots, 1$ verifies the identity.

fwd_zbase_range. For every legal forward length, $\text{fwd_zbase}(\ell)$ lies between 0 and 255.

Proof. Immediate from the explicit table 0, 1, 3, 7, 15, 31, 63, 127, 255.

fwd_read_range. Assume a positive legal length, a block start $0 \leq \text{start} < 256$, and

$$\text{start} = 2\ell(zc - \text{fwd_zbase}(\ell)).$$

Then the forward table read index $zc + 1$ satisfies $1 \leq zc + 1 < 256$.

Proof. The equation expresses $zc - \text{fwd_zbase}(\ell)$ as the number of completed blocks in the layer. Since start is a valid block start strictly below 256, this number lies in the range of blocks of the current layer. Combining this with the explicit fwd_zbase table gives $zc + 1 \in \{1, \dots, 255\}$.

fwd_block_end_bound. Under the hypotheses of fwd_read_range ,

$$\text{start} + 2\ell \leq 256.$$

Proof. The same block-count equation says that start is the beginning of a full block of size 2ℓ in a 256-entry array. Legal layer sizes divide 256, so the end of the block is still within the array.

fwd_stage_zbase_exit. If a positive forward layer exits the middle loop, so that

$$256 = 2\ell(zc - \text{fwd_zbase}(\ell)),$$

then $zc = \text{fwd_zbase}(\ell/2)$.

Proof. At layer exit, the number of consumed blocks is $256/(2\ell)$. Adding this quantity to the old zeta base gives the next zeta base by **fwd_zbase_next**.

fwd_middle_to_inner. The middle-loop bound at block start equals the inner-loop bound with $j = \text{start}$:

$$\text{fwd_middle_bound}(s, \text{start}, i) = \text{fwd_inner_bound}(s, \ell, \text{start}, \text{start}, i).$$

Proof. At the beginning of a block, no coefficient of that block has been processed yet. Expanding both definitions shows that the only entries assigned the larger bound are exactly those with $0 \leq i < \text{start}$.

fwd_inner_current. For j inside the current block, the current butterfly inputs j and $j + \ell$ have bound s , and after advancing to $j + 1$ both positions have bound $s + 1$.

Proof. Unfold **fwd_inner_bound**. Before processing the pair, neither j nor $j + \ell$ lies in the completed subranges. After incrementing j , both positions lie in completed subranges.

fwd_inner_weaken_unchanged. For any index k distinct from j and $j + \ell$, the inner bound at j is no larger than the inner bound at $j + 1$.

Proof. Advancing the inner loop only adds the two current positions to the processed sets. All other positions either keep the same status or move from smaller to larger bound. The result follows by unfolding the range predicates.

fwd_inner_exit_to_middle. At $j = \text{start} + \ell$, the inner-loop bound equals the middle-loop bound for the next block start $\text{start} + 2\ell$.

Proof. At inner-loop exit, both halves of the block have been processed. The union of the old completed prefix, the first half of the block, and the second half of the block is precisely the prefix $0 \leq i < \text{start} + 2\ell$.

fwd_middle_exit_const. If $\text{start} \geq 256$, then every in-range index has

$$\text{fwd_middle_bound}(s, \text{start}, i) = s + 1.$$

Proof. Every $i \in \{0, \dots, 255\}$ satisfies $i < \text{start}$, so the first branch of **fwd_middle_bound** applies.

fwd_middle_start_const. At $\text{start} = 0$,

$$\text{fwd_middle_bound}(s, 0, i) = s.$$

Proof. No index satisfies $0 \leq i < 0$, so the default branch of the definition applies.

8 Forward Butterfly and Twiddle Lemmas

forward_twiddle_product. Given a representation relation $rp \sim p$, a valid forward twiddle index zc , and operands $ai = \text{jzetas}[zc]$ and $bi = rp[j + \ell]$, a call to `__fqmul` returns a word representing

$$\text{zetas}[zc] \cdot p[j + \ell].$$

Proof. The representation relation gives

$$p[j + \ell] = \text{word}(bi).$$

The multiplication correctness lemma gives

$$\text{word}(res) = \text{word}(ai) \text{word}(bi) R^{-1}.$$

By `jzetas_get_cancel`, $\text{word}(ai) R^{-1} = \text{zetas}[zc]$. Associativity and commutativity in $\mathbf{F}_q$ yield the result.

forward_twiddle_product_bound_h. This strengthens `forward_twiddle_product` with the postcondition `bw32(res, 16)`.

Proof. The semantic part is `forward_twiddle_product`. The bound part is `fqmul_bw32_16`, using the same product-range hypothesis.

forward_twiddle_product_bound_ph. This is the probability-one phoare form of `forward_twiddle_product_bound_h`.

Proof. Combine losslessness of `__fqmul` with the ordinary Hoare triple.

forward_twiddle_product_bound_call_h. This is a call-friendly Hoare triple for `__fqmul`: all equalities tying local variables to the table entry and coefficient word are included in the precondition, and the postcondition gives both the abstract twiddle product and the 16-bit result bound.

Proof. Apply `fqmul_word_to_coeff_mul_bound_h`. Then rewrite the local-variable equalities, use `poly_repr_get` for $p[j + \ell]$, and use `jzetas_get_cancel` to remove the Montgomery factor.

forward_twiddle_product_bound_call_ph. The probability-one version of `forward_twiddle_product_bound_h`.

Proof. Use `Fq.fqmul_11` and the Hoare triple `forward_twiddle_product_bound_call_h`.

forward_twiddle_value. If a word t is known to satisfy

$$\text{word}(t) = \text{word}(ai) \text{word}(bi) R^{-1},$$

with $ai = \text{jzetas}[zc]$ and $bi = rp[j + \ell]$, then

$$\text{word}(t) = \text{zetas}[zc] p[j + \ell].$$

Proof. This is the pure post-call version of `forward_twiddle_product`. The representation relation identifies bi with $p[j + \ell]$, and `jzetas_get_cancel` identifies the table entry after multiplying by R^{-1} .

forward_butterfly_store. If $rp \sim p$, $s = rp[j]$, and the word operations $s - t$ and $s + t$ are within their signed ranges, then the two stores

$$rp[j + \ell] \leftarrow s - t, \quad rp[j] \leftarrow s + t$$

represent the polynomial update

$$p[j + \ell] \leftarrow p[j] - \text{word}(t), \quad p[j] \leftarrow p[j] + \text{word}(t).$$

Proof. The representation relation gives $p[j] = \text{word}(s)$. The lemmas `word_to_coeff_sub` and `word_to_coeff_add` convert the stored words to the corresponding field subtraction and addition. Two applications of `NTT_Fq.poly_repr_set32` complete the representation proof.

forward_butterfly_store_bound. Under a uniform s -bit input bound and a 16-bit bound on t , the two stores above produce a representation bounded by $\max(s, 16) + 1$.

Proof. The representation part is `forward_butterfly_store`. The bound part uses `forward_butterfly_bounds` for the two updated words, weakens all untouched lanes to the larger bound, and applies the uniform `barray256_bound_set32` lemmas.

forward_butterfly_store_bound_by. This is the pointwise-bound version of `forward_butterfly_store_bound`. It assumes that the current pair $j, j + \ell$ has bound a_s , and that the target bound bd' raises exactly those two positions to $a_s + 1$.

Proof. Use `forward_butterfly_store` for the representation. Use `forward_butterfly_bounds` for the two newly written words. The first store is handled by `barray256_bound_by_set32_change`; the second by `barray256_bound_by_set32`. The hypothesis on untouched lanes supplies all weakenings.

forward_spec_updateE. The two syntactic forms of the forward specification update are extensionally equal:

$$(p[j + \ell \leftarrow p[j] - t])[j \leftarrow (p[j + \ell \leftarrow p[j] - t])[j] + t] = (p[j + \ell \leftarrow p[j] - t])[j \leftarrow p[j] + t].$$

Proof. Since $\ell > 0$, the indices j and $j + \ell$ are distinct. Extensional array equality reduces the claim to an arbitrary index i ; the get-after-set rules split on whether i is j , $j + \ell$, or neither.

forward_butterfly_step_bound_by. One full forward butterfly step preserves representation and updates the pointwise inner-loop bound from `fwd_inner_bound`(s, ℓ, start, j) to `fwd_inner_bound`($s, \ell, \text{start}, j + 1$).

Proof. `fwd_inner_current` shows that the two current inputs have bound s . The 16-bit multiplication result is the t input to the store. Apply `forward_butterfly_store_bound_by`; the untouched-lane weakening is exactly `fwd_inner_weaken_unchanged`. Finally `forward_spec_updateE` aligns the array update with the right-hand imperative specification.

forward_twiddle_product_bound_from_inner. Inside the forward inner-loop invariant, the product

$$jzetas[zc] \cdot rp[j + \ell]$$

meets the signed Montgomery reducer input range.

Proof. `jzetas_bound16` gives the 16-bit twiddle bound. By `fwd_inner_current`, the current second butterfly input has bound `fwd_stage_bound` at length ℓ ; this bound is at most 24 by `fwd_stage_bound_range`. Weakening gives a 24-bit coefficient bound. Now apply `fqmul_product_bound_16_24`.

forward_butterfly_repr. If t already represents the abstract product $\text{zetas}[zc]p[j + \ell]$, then the two Jasmin stores represent the forward specification butterfly update.

Proof. Apply **forward_butterfly_store** and rewrite $\text{word}(t)$ to the abstract product.

forward_butterfly_step. If t satisfies the Montgomery product equation using the concrete forward twiddle and input word, then the two stores represent the abstract forward butterfly update.

Proof. First use **forward_twiddle_value** to convert the Montgomery product equation into the abstract product equation. Then apply **forward_butterfly_repr**.

9 Inverse Stage Arithmetic

The inverse stage definitions encode the ascending layer sequence:

$$1, 2, 4, 8, 16, 32, 64, 128, 256.$$

The predicate **inv_len_ok** is the finite disjunction recognizing exactly these legal loop lengths. The bound table is again $16, \dots, 24$, while the inverse zeta base table is

ℓ	1	2	4	8	16	32	64	128	256
$\text{inv_zbase}(\ell)$	0	128	192	224	240	248	252	254	255.

inv_stage_bound_range. For every legal inverse length, $\text{inv_stage_bound}(\ell)$ lies between 0 and 24.

Proof. Case analysis over the legal inverse lengths gives one of the values $16, \dots, 24$.

inv_stage_bound_pos. For every legal inverse length, $\text{inv_stage_bound}(\ell) \geq 0$.

Proof. This is immediate from **inv_stage_bound_range**.

inv_stage_bound_lt31. For every legal inverse length, $\text{inv_stage_bound}(\ell) < 31$.

Proof. **inv_stage_bound_range** gives an upper bound of 24.

inv_stage_bound_ge16. For every legal inverse length, $\text{inv_stage_bound}(\ell) \geq 16$.

Proof. Each case in the explicit definition is one of $16, \dots, 24$.

inv_stage_next. If $\ell < 256$ is a legal inverse length, then

$$\text{inv_stage_bound}(2\ell) = \text{inv_stage_bound}(\ell) + 1.$$

Proof. Doubling moves one step to the right in the inverse bound table.

inv_len_next_ok. If $\ell < 256$ is a legal inverse length, then 2ℓ is legal.

Proof. The legal values are powers of two from 1 through 256. Doubling a value below 256 gives the next value in the list.

inv_zbase_range. For every legal inverse length, $\text{inv_zbase}(\ell)$ lies between 0 and 255.

Proof. Immediate from the explicit inverse base table.

inv_read_range. Assume a legal inverse length $\ell < 256$, $0 \leq start < 256$, and

$$start = 2\ell (zc - \text{inv_zbase}(\ell)).$$

Then $zc \in \{0, \dots, 254\}$.

Proof. The equation identifies $zc - \text{inv_zbase}(\ell)$ with the completed block count in the current inverse layer. Since $start$ is a valid block start and $\ell < 256$, the layer consumes only indices below 255; checking the finite base table gives $zc \in \{0, \dots, 254\}$.

inv_block_end_bound. Under the same hypotheses,

$$start + 2\ell \leq 256.$$

Proof. The start position is a valid block beginning in a layer whose block size is 2ℓ , and legal inverse lengths divide 256.

inv_stage_zbase_exit. If an inverse layer exits with

$$256 = 2\ell (zc - \text{inv_zbase}(\ell)),$$

then $zc = \text{inv_zbase}(2\ell)$.

Proof. At layer exit, the number of consumed twiddle entries is the number of blocks in the layer. Adding that count to the old base gives the next base by the explicit inverse table.

inv_middle_to_inner. At block start, the inverse middle bound equals the inverse inner bound with $j = start$.

Proof. No coefficient of the new block has been processed yet; unfolding the two definitions leaves exactly the same processed prefix $0 \leq i < start$.

inv_inner_current. For j inside an inverse block, the current inputs j and $j + \ell$ have the old stage bound s , and after advancing to $j + 1$ both have bound $s + 1$.

Proof. The inverse inner bound has the same shape as the forward one. The current pair is outside the completed subranges before the step and inside them after the step.

inv_inner_weaken_unchanged. For indices other than the current pair, the inverse inner bound can only stay the same or increase when j advances to $j + 1$.

Proof. Unfold the definition. Advancing the loop extends the completed ranges by exactly the two current positions.

inv_inner_exit_to_middle. At inverse inner-loop exit, the inverse inner bound is the middle bound for the next block start.

Proof. At $j = start + \ell$, the entire block $start, \dots, start + 2\ell - 1$ has been processed. This is exactly the prefix extension represented by $\text{inv_middle_bound}(s, start + 2\ell, -)$.

inv_middle_exit_const. If $start \geq 256$, every in-range index has inverse middle bound $s + 1$.

Proof. All indices $0, \dots, 255$ lie below $start$, so they are in the completed prefix.

inv_middle_start_const. At $start = 0$, the inverse middle bound is the constant bound s .

Proof. The completed prefix is empty.

inverse_spec_updateE. The generated inverse update expression is extensionally equal to the compact mathematical update

$$(p[j \leftarrow p[j] + p[j + \ell]])[j + \ell \leftarrow z(p[j] - p[j + \ell])].$$

Proof. Since $\ell > 0$, the indices j and $j + \ell$ are distinct. Extensional array equality and get-after-set case analysis reduce both sides to the same value at every index.

10 Inverse Butterfly and Twiddle Lemmas

inverse_twiddle_product_bound_call_h. For a valid inverse twiddle index zc , if $ai = \text{jzetas_inv}[zc]$ and bi represents the field difference $diff$, and the signed product bound holds, then `__fpmul` returns a 16-bit word representing

$$\text{zetas_inv}[zc] \cdot diff.$$

Proof. `fpmul_word_to_coeff_mul_bound_h` gives

$$\text{word}(res) = \text{word}(ai) \text{word}(bi) R^{-1}$$

and a 16-bit result bound. Substitute $\text{word}(bi) = diff$, and use `jzetas_inv_get_cancel` to replace $\text{word}(ai)R^{-1}$ by $\text{zetas_inv}[zc]$.

inverse_twiddle_product_bound_call_ph. The probability-one version of `inverse_twiddle_product_bound`

Proof. Combine the ordinary Hoare triple with `Fq.fpmul_11`.

inverse_twiddle_value. If a word t satisfies the Montgomery product equation with an inverse twiddle and the field difference $p[j] - p[j + \ell]$, then

$$\text{word}(t) = \text{zetas_inv}[zc] (p[j] - p[j + \ell]).$$

Proof. Substitute $ai = \text{jzetas_inv}[zc]$ and use `jzetas_inv_get_cancel`. The remaining equality is associativity in $\mathbf{F}_q$.

inverse_butterfly_store. If $rp \sim p$, $t_0 = rp[j]$, $coeff = rp[j + \ell]$, and t represents the desired target value, then the stores

$$rp[j] \leftarrow t_0 + coeff, \quad rp[j + \ell] \leftarrow t$$

represent the inverse butterfly update

$$p[j] \leftarrow p[j] + p[j + \ell], \quad p[j + \ell] \leftarrow target.$$

Proof. The representation relation identifies t_0 and $coeff$ with $p[j]$ and $p[j + \ell]$. The lemma `word_to_coeff_add` converts $t_0 + coeff$ into the field sum. Two applications of `NTT_Fq.poly_repr_set32` prove the representation after the two stores.

inverse_butterfly_store_bound. Under a uniform s -bit bound and a 16-bit bound on t , the inverse butterfly stores produce a representation bounded by $\max(s, 16) + 1$.

Proof. The representation part is `inverse_butterfly_store`. The bound for $t_0 + coeff$ follows from `Fq.add_corr`; the bound for t is weakened from 16 bits. Untouched entries are weakened from s to the larger bound, and the two store-bound lemmas finish the proof.

inverse_butterfly_store_bound_by. This is the pointwise-bound version of the previous lemma. The two current inputs have bound a_s ; the target bound raises both updated positions to $a_s + 1$.

Proof. Use `inverse_butterfly_store` for representation and `inverse_butterfly_bounds` for the new sum bound. The freshly multiplied word is 16-bit and therefore weakens to $a_s + 1$. The two pointwise get-after-set lemmas handle the stores.

inverse_butterfly_step_bound_by. One full inverse butterfly step preserves representation and advances the inverse inner-loop bound from j to $j + 1$.

Proof. `inv_inner_current` supplies the bounds on the two inputs. `inverse_butterfly_store_bound_by` performs the store proof. The untouched-lane hypothesis is supplied by `inv_inner_weaken_unchanged`.

inverse_twiddle_product_bound_from_inner. Inside the inverse inner-loop invariant, the word difference

$$rp[j] - rp[j + \ell]$$

has bound `inv_stage_bound`(ℓ) + 1.

Proof. `inv_inner_current` gives the same stage bound for the two current input words. Applying `Fq.sub_corr` to those two bounded words gives one additional bit for the difference.

inverse_twiddle_product_call_bound_from_inner. Inside the inverse inner-loop invariant, the product of the inverse twiddle and the word difference lies in the signed Montgomery reducer input range.

Proof. The inverse twiddle is 16-bit by `jzetas_inv_bound16`. The difference is bounded by `inv_stage_bound` at length ℓ , plus one bit, by the previous lemma; for $\ell < 256$ this is at most 24. Weakening gives a 24-bit bound, and `fqmul_product_bound_16_24` supplies the reducer range.

inverse_butterfly_step_full. If t satisfies the Montgomery product equation for the inverse twiddle and the field difference, then the two stores represent the abstract inverse butterfly update.

Proof. Use `inverse_twiddle_value` to convert the Montgomery product equation into the abstract product with `zetas_inv[zc]`. Then apply `inverse_butterfly_store`.

inverse_butterfly_repr. This is another representation-only form of the inverse butterfly step, using the same hypotheses as `inverse_butterfly_step_full`.

Proof. The proof is identical in structure: derive the abstract value of t by `inverse_twiddle_value`, then call `inverse_butterfly_store`.

11 Final Inverse Scaling Lemmas

The final inverse loop is tracked by the ghost array

$$\text{final_mont_array}(p, j)[i] = \begin{cases} p[i]R, & 0 \leq i < j, \\ p[i], & \text{otherwise.} \end{cases}$$

The matching bound function is

$$\text{final_bound}(j, i) = \begin{cases} 16, & 0 \leq i < j, \\ 24, & \text{otherwise.} \end{cases}$$

final_mont_array_start.

$$\text{final_mont_array}(p, 0) = p.$$

Proof. The prefix $0 \leq i < 0$ is empty, so the initializer returns $p[i]$ at every index.

final_mont_array_exit.

$$\text{final_mont_array}(p, 256) = \text{NTT_Fq.array256_mont}(p).$$

Proof. Every array index lies in the prefix $0 \leq i < 256$. Hence the initializer returns $p[i]R$ at each index, which is exactly `array256_mont` by `array256_mont_get`.

final_mont_array_get_raw. If $0 \leq j < 256$, then the j -th entry of `final_mont_array`(p, j) is still $p[j]$.

Proof. The condition for Montgomery scaling is $0 \leq i < j$. At $i = j$, this condition is false, so the raw entry $p[j]$ is returned.

final_mont_array_setE. For $0 \leq j < 256$, storing $p[j] \cdot \text{scale255} \cdot R$ at index j of `final_mont_array`(p, j) yields

$$\text{final_mont_array}(p[j \leftarrow p[j] \cdot \text{scale255}], j + 1).$$

Proof. Use extensional equality. At $i = j$, both sides are the newly scaled value. For $i < j$, both sides are already Montgomery-scaled. For $i > j$, both sides are still raw entries. The get-after-set rules separate these cases.

final_bound_start.

$$\text{final_bound}(0, i) = 24.$$

Proof. No index lies in the processed prefix $0 \leq i < 0$.

final_bound_next_unchanged. If $k \neq j$, then the bound at k under `final_bound`($j, -$) is no larger than the bound under `final_bound`($j + 1, -$).

Proof. Advancing j changes only the status of index j , moving it from the 24-bit unprocessed region to the 16-bit processed region. Every other index keeps the same status, or is already in the processed prefix.

inverse_final_product_bound_call_h. For the special table entry $ai = \text{jzetas_inv}[255]$, if bi represents *coeff* and the signed product bound holds, then `__fqmul` returns a 16-bit word representing

$$\text{coeff} \cdot \text{scale255} \cdot R.$$

Proof. `fqmul_word_to_coeff_mul_bound_h` gives the Montgomery product $\text{word}(ai) \text{word}(bi)R^{-1}$ and the 16-bit bound. The hypothesis identifies $\text{word}(bi) = \text{coeff}$, and `jzetas_inv_255_scale_cancel` rewrites $\text{word}(ai)R^{-1}$ as `scale255R`.

inverse_final_product_bound_call_ph. The probability-one version of `inverse_final_product_bound_call`

Proof. Use losslessness of `__fqmul` and the ordinary Hoare triple.

inverse_final_product_bound_from_inv. Under the final-loop bound invariant, the product of `jzetas_inv[255]` and the current unprocessed coefficient word lies in the signed Montgomery reducer input range.

Proof. `jzetas_inv_bound16` gives the 16-bit bound for the scale table entry. The current unprocessed coefficient has 24-bit bound by `final_bound`. Apply `fqmul_product_bound_16_24`.

12 Wrapper and Core pRHL Equivalences

poly_ntt_jazz_ref. The public extracted wrapper `poly_ntt_jazz` is equivalent to the core procedure `_poly_ntt` under equality of the input buffer and equality of the result.

Proof. The wrapper contains only the core call. Inlining all procedures reduces both sides to the same program, and pRHL simulation proves equality of results.

poly_invntt_jazz_ref. The public extracted wrapper `poly_invntt_jazz` is equivalent to the core procedure `_poly_invntt`.

Proof. As in the forward case, inlining exposes a direct call to the core routine, and the two sides simulate identically.

poly_ntt_core_ref. The extracted forward core refines the imperative EasyCrypt forward NTT:

$$\begin{aligned} & \text{Hpoly_extract.M.}_poly_ntt \sim \text{NTT_Fq.NTT.ntt}, \\ & \text{poly_repr_bound}(rp_1, r_2, 16) \wedge \text{zetas}_2 = \text{NTT_Fq.zetas} \\ & \implies \text{poly_repr_bound}(res_1, res_2, 24). \end{aligned}$$

Proof. The proof is a direct three-loop pRHL simulation. The outer invariant aligns the layer length, twiddle counter, table pointers, representation relation, and a uniform pointwise bound `fwd_stage_bound( $\ell$ )`. The middle invariant refines the bound to `fwd_middle_bound`, distinguishing completed blocks from untouched blocks. The inner invariant refines it further to `fwd_inner_bound`, distinguishing completed butterflies inside the current block.

At each inner iteration, `forward_twiddle_product_bound_call_ph` verifies the call to `__fqmul`; its product-range precondition is supplied by `forward_twiddle_product_bound_from_inner`. The subsequent stores are verified by `forward_butterfly_step_bound_by`. Inner-loop exit uses `fwd_inner_exit_to_middle`; middle-loop exit uses `fwd_middle_exit_const`; outer-loop advancement uses `fwd_stage_next`, `fwd_len_next_ok`, and `fwd_zbase_next`. At outer-loop exit, $\ell = 0$, so the bound is 24, giving the stated postcondition.

poly_invntt_core_ref. The extracted inverse core refines the imperative EasyCrypt inverse NTT:

$$\begin{aligned} & \text{Hpoly_extract.M.}_poly_invntt \sim \text{NTT_Fq.NTT.invntt}, \\ & \text{poly_repr_bound}(rp_1, r_2, 16) \wedge \text{zetas_inv}_2 = \text{NTT_Fq.zetas_inv} \\ & \implies \text{poly_repr_bound}(res_1, \text{NTT_Fq.array256_mont}(res_2), 16). \end{aligned}$$

Proof. The inverse proof has two phases. First, weakest-precondition reasoning encounters the syntactically final scaling loop. Its invariant represents the left buffer as `final_mont_array` of the right buffer and uses `final_bound` to say that the processed prefix is 16-bit while the unprocessed suffix is 24-bit. Each iteration is justified by `inverse_final_product_bound_call_ph` and `inverse_final_product_bound_from_inv`. The lemmas `final_mont_array_setE`,

`final_bound_next_unchanged`, and `final_mont_array_exit` carry the loop from an empty Montgomery prefix to `array256_mont`.

Second, the proof establishes the eight inverse butterfly layers with the same three-loop structure as the forward proof, using `inv_stage_bound`, `inv_middle_bound`, and `inv_inner_bound`. The multiplication call inside a butterfly is handled by `inverse_twiddle_product_bound_call_ph`; the product-range side condition comes from `inverse_twiddle_product_call_bound_from_inner`. The stores are handled by `inverse_butterfly_step_bound_by`, and `inverse_spec_updateE` aligns the right-hand array update with the generated program expression. Layer advancement uses `inv_stage_next`, `inv_len_next_ok`, and `inv_stage_zbase_exit`. After the butterfly layers, the output is represented with a 24-bit bound; this initializes the final scaling invariant. The final loop then reduces every entry back to 16 bits and produces the Montgomery-scaled representation required by the theorem.

13 Conclusion

Every lemma in `RefJasminNTT.ec` serves one of four mathematical roles:

1. interpreting machine words as field elements without overflow;
2. cancelling Montgomery factors in concrete twiddle tables;
3. maintaining pointwise word-size bounds through butterfly loops;
4. assembling those local facts into direct pRHL simulations of the extracted forward and inverse NTT cores.

The final result is a checked refinement layer from Jasmin extraction to the imperative EasyCrypt NTT model, with the inverse postcondition strengthened to the mathematically correct Montgomery-scaled representation.